1. What is RIGHT JOIN?

`RIGHT JOIN` returns:

- All records from the right table
- Matching records from the left table
- If no match is found in the left table, left table columns come as `NULL`

Simple meaning:

RIGHT JOIN = Keep all rows from the right table and bring matching data from the left table

Example:

orders RIGHT JOIN customers

This means:

Show all customers and their orders if available

Here, `customers` is on the right side, so all customers will come.

---

2. LEFT JOIN vs RIGHT JOIN

LEFT JOIN

Keeps all rows from the left table.

customers LEFT JOIN orders

Meaning:

All customers + matching orders

RIGHT JOIN

Keeps all rows from the right table.

orders RIGHT JOIN customers

Meaning:

All customers + matching orders

Both queries can give the same result.

That is why in production, `LEFT JOIN` is usually preferred because it is easier to read.

---

3. Why RIGHT JOIN Is Rarely Used

`RIGHT JOIN` is valid SQL, but it is not used very frequently.

Reasons:

- `LEFT JOIN` is easier to read.
- Developers usually keep the base table first.
- `RIGHT JOIN` can confuse the query flow.
- Most `RIGHT JOIN` queries can be rewritten using `LEFT JOIN`.
- In multi-table joins, `LEFT JOIN` looks cleaner.

Example:

RIGHT JOIN:

```
SELECT
   c.customer_name,
   o.order_id
FROM orders o
RIGHT JOIN customers c
   ON o.customer_id = c.customer_id;
```

Same query using LEFT JOIN:

```
SELECT
    c.customer_name,
    o.order_id
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id;
```

Both mean:

Show all customers and their orders if available

---

4. Complete SQL Setup

Use this setup from scratch.

```
DROP DATABASE IF EXISTS joins_practice;

CREATE DATABASE joins_practice;

USE joins_practice;
```

---

5. Create Tables

customers table

```
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(50) NOT NULL,
    city VARCHAR(50),
    signup_date DATE
);
```

---

products table

```
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    category VARCHAR(50)
);
```

---

orders table

```
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    product_id INT,
    order_amount DECIMAL(10,2),
    order_date DATE,
    order_status VARCHAR(30)
);
```

---

payments table

```
CREATE TABLE payments (
    payment_id INT PRIMARY KEY,
    order_id INT,
    payment_mode VARCHAR(30),
    payment_status VARCHAR(30),
    paid_amount DECIMAL(10,2),
    payment_date DATE
);
```

Note:

Foreign key constraints are not added intentionally.
This helps us insert some invalid records and understand unmatched data clearly.

---

6. Insert Sample Data

Insert customers

```
INSERT INTO customers
(customer_id, customer_name, city, signup_date)
VALUES
(1, 'Anurag', 'Bengaluru', '2026-01-10'),
(2, 'Stuti', 'Pune', '2026-02-15'),
(3, 'Rahul', 'Delhi', '2026-03-12'),
(4, 'Priya', 'Mumbai', '2026-04-05'),
(5, 'Neha', 'Bengaluru', '2026-05-20'),
(6, 'Aman', 'Hyderabad', '2026-06-10');
```

Insert products

```
INSERT INTO products
(product_id, product_name, category)
VALUES
(201, 'Laptop', 'Electronics'),
(202, 'Mouse', 'Accessories'),
(203, 'Keyboard', 'Accessories'),
(204, 'Monitor', 'Electronics'),
(205, 'Webcam', 'Accessories');
```

Insert orders

```
INSERT INTO orders
(order_id, customer_id, product_id, order_amount, order_date, order_status)
VALUES
(101, 1, 201, 50000.00, '2026-06-01', 'Delivered'),
(102, 1, 202, 1000.00, '2026-06-02', 'Delivered'),
(103, 2, 203, 2500.00, '2026-06-03', 'Delivered'),
(104, 99, 204, 15000.00, '2026-06-04', 'Delivered'),
(105, 3, 999, 3500.00, '2026-06-05', 'Delivered'),
(106, NULL, 201, 45000.00, '2026-06-06', 'Pending'),
(107, 5, 202, 1200.00, '2026-06-07', 'Cancelled'),
(108, 2, 201, 48000.00, '2026-06-08', 'Delivered');
```

Important observations:

- Anurag has 2 orders.

- Stuti has 2 orders.
- Rahul has 1 order with invalid product ID.
- Priya has no order.
- Aman has no order.
- Order 104 has invalid customer ID `99`.
- Order 105 has invalid product ID `999`.
- Order 106 has `customer_id = NULL`.
- Order 107 has no payment.

Insert payments

INSERT INTO payments
(payment_id, order_id, payment_mode, payment_status, paid_amount, payment_date)
VALUES
(1001, 101, 'UPI', 'Success', 50000.00, '2026-06-01'),
(1002, 102, 'Card', 'Success', 1000.00, '2026-06-02'),
(1003, 103, 'UPI', 'Failed', 0.00, '2026-06-03'),
(1004, 105, 'Wallet', 'Success', 3500.00, '2026-06-05'),
(1005, 999, 'UPI', 'Success', 3000.00, '2026-06-08'),
(1006, 108, 'Card', 'Success', 48000.00, '2026-06-08');

Important observations:

- Payment 1001 belongs to order 101.
- Payment 1002 belongs to order 102.
- Payment 1003 belongs to order 103, but payment failed.
- Payment 1004 belongs to order 105.
- Payment 1005 belongs to invalid order ID 999.
- Payment 1006 belongs to order 108.
- Order 107 has no payment.

---

7. Check the Data

SELECT * FROM customers;
SELECT * FROM products;
SELECT * FROM orders;
SELECT * FROM payments;

Check row counts:

```
SELECT COUNT(*) AS customer_count FROM customers;
SELECT COUNT(*) AS product_count FROM products;
SELECT COUNT(*) AS order_count FROM orders;
SELECT COUNT(*) AS payment_count FROM payments;
```

Expected counts:

customers = 6
products = 5
orders = 8
payments = 6

---

8. Basic RIGHT JOIN Syntax

```
SELECT
   table1.column_name,
   table2.column_name
FROM table1
RIGHT JOIN table2
   ON table1.common_column = table2.common_column;
```

Important:

RIGHT JOIN always protects the right table.

So the table after `RIGHT JOIN` is very important.

Level 1: Basic RIGHT JOIN Questions

Question 1

Show all customers and their orders using RIGHT JOIN.

Solution

```
SELECT
    c.customer_id,
    c.customer_name,
    c.city,
    o.order_id,
    o.order_amount,
    o.order_status
FROM orders o
RIGHT JOIN customers c
    ON o.customer_id = c.customer_id;
```

Explanation

- `customers` is the right table.
- RIGHT JOIN keeps all customers.
- Priya and Aman have no orders, so order columns will be `NULL`.

Question 2

Show customer name and order ID using RIGHT JOIN.

Solution

```
SELECT
    c.customer_name,
    o.order_id
FROM orders o
RIGHT JOIN customers c
    ON o.customer_id = c.customer_id;
```

Explanation

All customers will come because `customers` is on the right side.

If one customer has multiple orders, that customer appears multiple times.

Question 3

Show all products and their orders using RIGHT JOIN.

Solution

```
SELECT
   p.product_id,
   p.product_name,
   p.category,
   o.order_id,
   o.order_amount
FROM orders o
RIGHT JOIN products p
   ON o.product_id = p.product_id;
```

Explanation

- All products will come.
- Product 205, Webcam, has no order.
- So order columns will be `NULL`.

Question 4

Show all orders and payment details using RIGHT JOIN.

Solution

```
SELECT
   o.order_id,
   o.order_amount,
   o.order_status,
   pay.payment_id,
   pay.payment_status,
   pay.paid_amount
FROM payments pay
RIGHT JOIN orders o
   ON pay.order_id = o.order_id;
```

Explanation

- `orders` is the right table.
- All orders will come.
- Orders without payment will show `NULL` in payment columns.

Level 2: Missing Records Questions

Question 5

Find customers who never placed any order using RIGHT JOIN.

Solution

```
SELECT
   c.customer_id,
   c.customer_name,
   c.city
FROM orders o
RIGHT JOIN customers c
   ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;
```

Expected Result

Priya
Aman

Explanation

RIGHT JOIN keeps all customers.

Customers with no matching order will have `NULL` in order columns.

So we filter:

WHERE o.order_id IS NULL

Question 6

Find products that were never ordered using RIGHT JOIN.

Solution

```
SELECT
    p.product_id,
    p.product_name,
    p.category
FROM orders o
RIGHT JOIN products p
    ON o.product_id = p.product_id
WHERE o.order_id IS NULL;
```

Expected Result

Webcam

Explanation

Webcam exists in products but does not exist in orders.

So order columns are `NULL`.

Question 7

Find orders where payment is not available using RIGHT JOIN.

Solution

```
SELECT
    o.order_id,
    o.customer_id,
    o.order_amount,
    o.order_status
FROM payments pay
RIGHT JOIN orders o
    ON pay.order_id = o.order_id
WHERE pay.payment_id IS NULL;
```

Expected Result

Order 104
Order 106
Order 107

Explanation

These orders do not have matching payment records.

Level 3: Convert RIGHT JOIN to LEFT JOIN

Question 8

Rewrite this RIGHT JOIN query as LEFT JOIN.

RIGHT JOIN Query

```
SELECT
   c.customer_name,
   o.order_id
FROM orders o
RIGHT JOIN customers c
   ON o.customer_id = c.customer_id;
```

LEFT JOIN Solution

```
SELECT
   c.customer_name,
   o.order_id
FROM customers c
LEFT JOIN orders o
   ON c.customer_id = o.customer_id;
```

Explanation

Both queries mean:

All customers + matching orders

Question 9

Rewrite this RIGHT JOIN query as LEFT JOIN.

RIGHT JOIN Query

```
SELECT
    p.product_name,
    o.order_id
FROM orders o
RIGHT JOIN products p
    ON o.product_id = p.product_id;
```

LEFT JOIN Solution

```
SELECT
    p.product_name,
    o.order_id
FROM products p
LEFT JOIN orders o
    ON p.product_id = o.product_id;
```

Question 10

Rewrite this RIGHT JOIN query as LEFT JOIN.

RIGHT JOIN Query

```
SELECT
    o.order_id,
    pay.payment_status
FROM payments pay
RIGHT JOIN orders o
    ON pay.order_id = o.order_id;
```

LEFT JOIN Solution

```
SELECT
   o.order_id,
   pay.payment_status
FROM orders o
LEFT JOIN payments pay
   ON o.order_id = pay.order_id;
```

Level 4: Reporting Questions

Question 11

Show every customer with total order amount using RIGHT JOIN.

If customer has no order, show 0.

Solution

```
SELECT
   c.customer_id,
   c.customer_name,
   COALESCE(SUM(o.order_amount), 0) AS total_order_amount
FROM orders o
RIGHT JOIN customers c
   ON o.customer_id = c.customer_id
GROUP BY
   c.customer_id,
   c.customer_name;
```

Expected Logic

Anurag = 51000
Stuti = 50500
Rahul = 3500
Priya = 0
Neha = 1200
Aman = 0

Question 12

Show every customer with total number of orders using RIGHT JOIN.

Solution

```
SELECT
   c.customer_id,
   c.customer_name,
   COUNT(o.order_id) AS total_orders
FROM orders o
RIGHT JOIN customers c
   ON o.customer_id = c.customer_id
GROUP BY
   c.customer_id,
   c.customer_name;
```

Expected Logic

Anurag = 2
Stuti = 2
Rahul = 1
Priya = 0
Neha = 1
Aman = 0

Question 13

Show all products with total sales amount using RIGHT JOIN.

If product has no sales, show 0.

Solution

```
SELECT
   p.product_id,
   p.product_name,
   p.category,
   COALESCE(SUM(o.order_amount), 0) AS total_sales
FROM orders o
```

```
RIGHT JOIN products p
    ON o.product_id = p.product_id
GROUP BY
    p.product_id,
    p.product_name,
    p.category;
```

Expected Logic

- Laptop has orders 101, 106, and 108.
- Mouse has orders 102 and 107.
- Keyboard has order 103.
- Monitor has order 104.
- Webcam has no order, so total sales = 0.

Question 14

Show all orders with payment status. If payment is missing, show `No Payment`.

Solution

```
SELECT
    o.order_id,
    o.order_amount,
    o.order_status,
    COALESCE(pay.payment_status, 'No Payment') AS payment_status
FROM payments pay
RIGHT JOIN orders o
    ON pay.order_id = o.order_id;
```

Explanation

Orders without payment records will show `No Payment`.

Level 5: Common Mistakes and Tricky Concepts

Question 15

What is wrong with this query?

```
SELECT
   c.customer_name,
   COUNT(*) AS total_orders
FROM orders o
RIGHT JOIN customers c
   ON o.customer_id = c.customer_id
GROUP BY c.customer_name;
```

Answer

This query uses `COUNT(*)`.

In RIGHT JOIN, all customers are preserved.

`COUNT(*)` counts the customer row also, even if the customer has no order.

So Priya and Aman may show count 1 even though they have no orders.

Correct Solution

```
SELECT
   c.customer_name,
   COUNT(o.order_id) AS total_orders
FROM orders o
RIGHT JOIN customers c
   ON o.customer_id = c.customer_id
GROUP BY c.customer_name;
```

Rule

When counting matched records, count the matching table column, not COUNT(*).

Here, customers are preserved and orders are matched, so count `o.order_id`.

Question 16

Show all customers and only delivered orders if available.

Wrong Query

```
SELECT
   c.customer_name,
   o.order_id,
   o.order_status
FROM orders o
RIGHT JOIN customers c
   ON o.customer_id = c.customer_id
WHERE o.order_status = 'Delivered';
```

Problem

The `WHERE` condition removes customers where order columns are `NULL`.

So Priya and Aman disappear.

This makes RIGHT JOIN behave like INNER JOIN for this condition.

Correct Solution

```
SELECT
   c.customer_name,
   o.order_id,
   o.order_status
FROM orders o
RIGHT JOIN customers c
   ON o.customer_id = c.customer_id
  AND o.order_status = 'Delivered';
```

Rule

In RIGHT JOIN, if you want to preserve all right table rows, put left table filters inside ON.

Question 17

Why does table order matter in RIGHT JOIN?

Example 1

```
SELECT
   c.customer_name,
   o.order_id
FROM orders o
RIGHT JOIN customers c
   ON o.customer_id = c.customer_id;
```

Meaning:

All customers + matching orders

Example 2

```
SELECT
   c.customer_name,
   o.order_id
FROM customers c
RIGHT JOIN orders o
   ON c.customer_id = o.customer_id;
```

Meaning:

All orders + matching customers

Answer

Both queries are different because RIGHT JOIN always preserves the right table.

Question 18

Can RIGHT JOIN increase rows?

Answer

Yes.

RIGHT JOIN can increase rows if the left table has multiple matches for one right table row.

Example:

Anurag has 2 orders.

So Anurag appears 2 times in this query:

```
SELECT
   c.customer_name,
   o.order_id
FROM orders o
RIGHT JOIN customers c
   ON o.customer_id = c.customer_id
WHERE c.customer_name = 'Anurag';
```

Expected:

Anurag - 101
Anurag - 102

Level 6: Multi-Table RIGHT JOIN Questions

Question 19

Show all customers with order and payment details wherever available using RIGHT JOIN.

Solution

```
SELECT
   c.customer_id,
   c.customer_name,
   o.order_id,
   o.order_amount,
   pay.payment_status
FROM payments pay
RIGHT JOIN orders o
   ON pay.order_id = o.order_id
RIGHT JOIN customers c
```

```
    ON o.customer_id = c.customer_id;
```

Note

This query is valid, but it is not very readable.

Preferred LEFT JOIN version:

```
SELECT
    c.customer_id,
    c.customer_name,
    o.order_id,
    o.order_amount,
    pay.payment_status
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
LEFT JOIN payments pay
    ON o.order_id = pay.order_id;
```

Question 20

Show all customers with order, product, and payment details using the preferred LEFT JOIN version.

Solution

```
SELECT
    c.customer_id,
    c.customer_name,
    c.city,
    o.order_id,
    p.product_name,
    o.order_amount,
    pay.payment_status
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
LEFT JOIN products p
    ON o.product_id = p.product_id
LEFT JOIN payments pay
    ON o.order_id = pay.order_id;
```

Explanation

This is usually better than writing multiple RIGHT JOINs.

Base table is clear: `customers`.

Level 7: Interview Questions and Answers

Q1. What is RIGHT JOIN?

RIGHT JOIN returns all records from the right table and matching records from the left table.

If no match is found in the left table, left table columns come as `NULL`.

Q2. What is the difference between LEFT JOIN and RIGHT JOIN?

LEFT JOIN preserves the left table.

RIGHT JOIN preserves the right table.

Example:

customers LEFT JOIN orders = all customers
orders RIGHT JOIN customers = all customers

Q3. Is RIGHT JOIN commonly used in production?

RIGHT JOIN is valid SQL, but it is not commonly preferred.

Most teams prefer LEFT JOIN because it keeps the base table first and makes the query easier to read.

Q4. Can RIGHT JOIN be converted into LEFT JOIN?

Yes.

Most RIGHT JOIN queries can be rewritten as LEFT JOIN by changing the table order.

Example:

```
FROM orders o
RIGHT JOIN customers c
    ON o.customer_id = c.customer_id;
```

Can be rewritten as:

```
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id;
```

Q5. Does table order matter in RIGHT JOIN?

Yes.

RIGHT JOIN preserves the table written on the right side.

So table order is very important.

Q6. Can RIGHT JOIN return more rows than the right table?

Yes.

If the left table has multiple matching rows for one right table row, output can be more than the right table count.

Example:

One customer has multiple orders.

So that customer appears multiple times.

Q7. Can RIGHT JOIN return fewer rows than the right table?

Normally, RIGHT JOIN keeps all rows from the right table.

But if a `WHERE` condition filters the final output, rows can become fewer.

Example:

WHERE o.order_status = 'Delivered'

This can remove customers who have no orders.

Q8. Why does RIGHT JOIN sometimes behave like INNER JOIN?

This happens when a filter is applied on the left table in the `WHERE` clause.

Rows with `NULL` from the left table are removed.

So unmatched right table records disappear.

Q9. What is the difference between filter in ON and WHERE for RIGHT JOIN?

Filter in `ON` controls which left table records should match.

Filter in `WHERE` filters the final result.

In RIGHT JOIN, left table filters in `WHERE` can remove unmatched right table rows.

Q10. Why is LEFT JOIN preferred over RIGHT JOIN?

LEFT JOIN is preferred because:

- It is easier to read.
- Base table comes first.
- It is more common in coding standards.
- It reduces confusion.
- RIGHT JOIN can usually be rewritten as LEFT JOIN.

Q11. Business asks: “Show all customers even if they have not ordered.” How can you write it using RIGHT JOIN?

```
SELECT
   c.customer_id,
   c.customer_name,
   o.order_id
FROM orders o
RIGHT JOIN customers c
   ON o.customer_id = c.customer_id;
```

Preferred version:

```
SELECT
   c.customer_id,
   c.customer_name,
   o.order_id
FROM customers c
LEFT JOIN orders o
   ON c.customer_id = o.customer_id;
```

Q12. Business asks: “Find products that were never ordered.” How can you write it using RIGHT JOIN?

```
SELECT
   p.product_id,
   p.product_name
```

```
FROM orders o
RIGHT JOIN products p
   ON o.product_id = p.product_id
WHERE o.order_id IS NULL;
```

Q13. Business asks: “Find orders without payments.” How can you write it using RIGHT JOIN?

```
SELECT
   o.order_id,
   o.order_amount
FROM payments pay
RIGHT JOIN orders o
   ON pay.order_id = o.order_id
WHERE pay.payment_id IS NULL;
```

Q14. What is the biggest mistake in RIGHT JOIN?

The biggest mistake is applying a filter on the left table inside the `WHERE` clause.

This can remove unmatched right table rows and make RIGHT JOIN behave like INNER JOIN.

Q15. Should we use RIGHT JOIN in production?

RIGHT JOIN is not wrong.

But in most cases, it is better to rewrite it as LEFT JOIN for readability.

---

8. Final Revision Points

- RIGHT JOIN keeps all records from the right table.
- Matching records from the left table are added.
- If no match is found, left table columns become `NULL`.

- Table order matters in RIGHT JOIN.
- RIGHT JOIN is less commonly used than LEFT JOIN.
- Most RIGHT JOIN queries can be rewritten as LEFT JOIN.
- RIGHT JOIN can return more rows than the right table if multiple matches exist.
- Be careful with filters on the left table in the `WHERE` clause.
- Use `COUNT(matching_table.column)`, not `COUNT(*)`, when counting matches.
- For production queries, LEFT JOIN is usually easier to read.

---

9. One-Line Summary

RIGHT JOIN keeps all rows from the right table, but in real projects we usually rewrite it as LEFT JOIN for better readability.